



Game Playing AI

CS 541 Lecture 5

Zining Zhu



Games as adversarial search

Minimax · Alpha-beta pruning

Games as a search problem

A game is a search problem with an **adversary** choosing some of the moves.

- State: the board / position
- Players alternate moves; each has **opposing** objectives
- Terminal states carry a **utility** (win/loss/draw, or a score)
- Goal: choose a move that is best **assuming the opponent plays well too**

Chess, Go, tic-tac-toe, and poker all fit this template, though poker adds hidden information.



Formalizing a game

A game is defined the same way a search problem is, plus one new ingredient.

- S_0 : the initial state
- $\text{Player}(s)$: whose turn it is at state s
- $\text{Actions}(s)$: legal moves at s
- $\text{Result}(s, a)$: the state after taking action a in state s
- $\text{Terminal-Test}(s)$: is the game over?
- $\text{Utility}(s, p)$: numeric payoff to player p at a terminal state

A **zero-sum** two-player game has $\text{Utility}(s, \text{MAX}) = -\text{Utility}(s, \text{MIN})$: one side's gain is exactly the other's loss.



A short history of game-playing AI

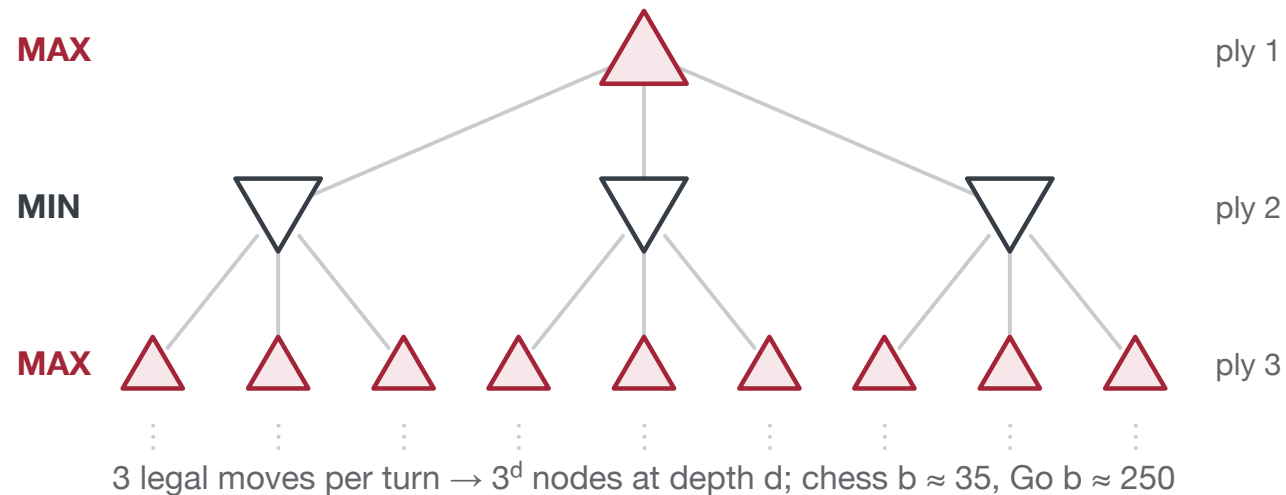
Year	Milestone
------	-----------

1950s	Early checkers and chess programs (Samuel, Turing, Shannon)
1997	Deep Blue beats world champion Garry Kasparov at chess
2011	IBM Watson wins Jeopardy! against human champions
2016	AlphaGo beats Lee Sedol at Go, 4 games to 1
2017	AlphaZero masters chess, shogi, and Go from self-play alone
2019	Superhuman poker (Pluribus) and Diplomacy-playing agents handle hidden information and negotiation



The game tree

- Nodes alternate between **MAX** (us) and **MIN** (opponent) layers
- Depth of the tree = number of **ply** (half-moves)
- Branching factor b and depth d give a tree of size roughly b^d
 - Chess: $b \approx 35$, Go: $b \approx 250$: this is why brute force fails fast
- Leaves are terminal states, each carrying a utility value



Minimax: the idea

Assume the opponent always plays their best response.

- At a MAX node: take the **maximum** of children's values
- At a MIN node: take the **minimum** of children's values
- At a terminal node: the value is the **utility**
- Recurse from the leaves upward: this is the **minimax value**



Minimax: the algorithm

- Depth-first traversal of the game tree
- `minimax(state)`:
 - if terminal: return `utility(state)`
 - if MAX's turn: return max over actions of `minimax(result(state, action))`
 - if MIN's turn: return min over actions of `minimax(result(state, action))`
- Time complexity: $O(b^d)$; space: $O(bd)$ (DFS)

Minimax gives the **optimal** move against an optimal opponent, but it is too slow to run to the bottom of most real game trees.



Minimax: complexity in practice

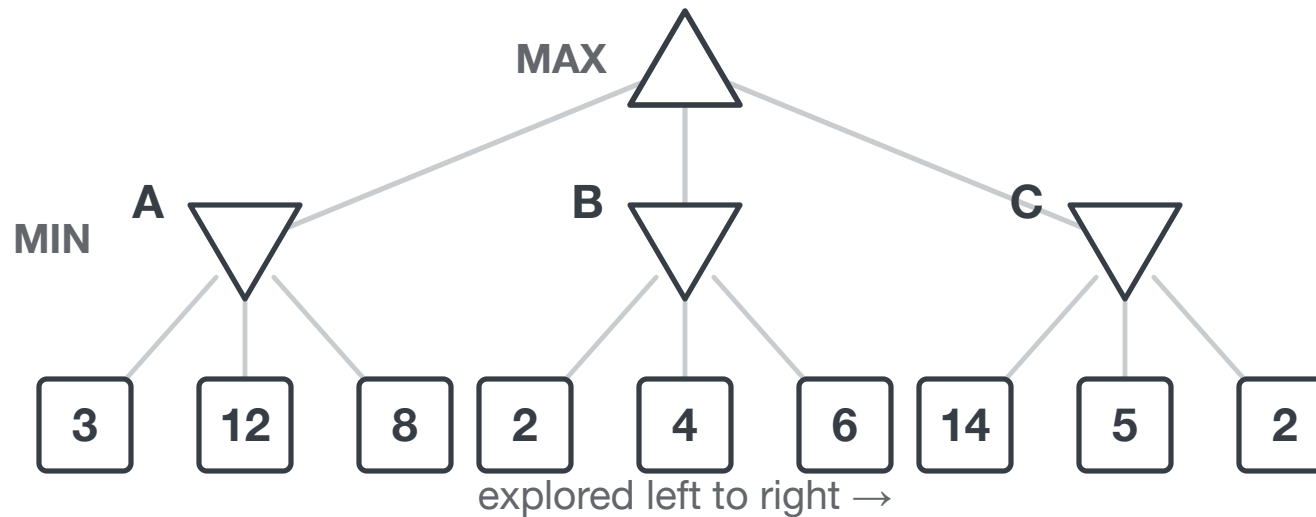
- Chess, $b \approx 35$: even 4 ply deep is $35^4 \approx 1.5$ million leaf evaluations
- A full game to checkmate can run 40-80 ply: fully solving minimax from the opening is hopeless
- Two escape hatches, both explored today: **prune** branches that can't matter (alpha-beta), or **sample** instead of enumerate (MCTS)



Worked example: the game tree we'll trace

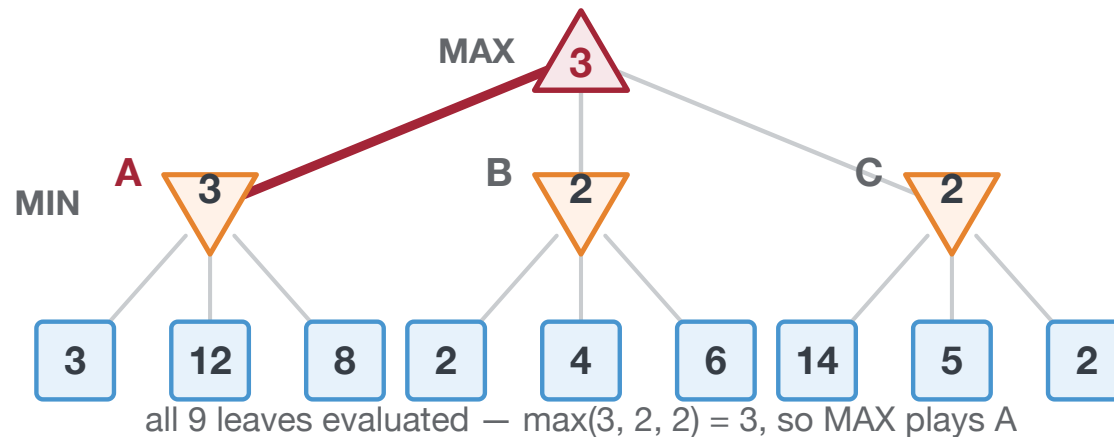
We'll trace minimax and alpha-beta on the same small tree for the rest of this section.

- **Root** (MAX) has three MIN children: **A**, **B**, **C**
- Each MIN node has three terminal children carrying utilities
- Children are always explored left to right: first child, then second, then third



Worked minimax trace: computing the value

Node	Children (left to right)	Minimax value
A (MIN)	3, 12, 8	$\min(3, 12, 8) = 3$
B (MIN)	2, 4, 6	$\min(2, 4, 6) = 2$
C (MIN)	14, 5, 2	$\min(14, 5, 2) = 2$
Root (MAX)	A=3, B=2, C=2	$\max(3, 2, 2) = 3$, choose A



Every leaf had to be visited: 9 terminal evaluations for this tiny tree.

Alpha-beta pruning: the idea

We don't need to fully explore a branch once we know it cannot change the parent's decision.

- Track two bounds while searching: α (best value MAX can guarantee so far, from the root's side) and β (best value MIN can guarantee so far, from the current branch's side)
- If a node's value would fall outside $[\alpha, \beta]$, **prune**: stop exploring its children
- Key property: alpha-beta returns the **exact same** minimax value, just faster



Alpha-beta pruning: the two bounds

- α : the best (highest) value MAX has found **anywhere so far** on the path to the root: a lower bound on the root's true value
- β : the best (lowest) value MIN has found **anywhere so far** on the current path: an upper bound on this branch's value
- Passed down the recursion, updated as search proceeds; each node inherits its parent's current $[\alpha, \beta]$ window



Alpha-beta pruning: the pruning rule

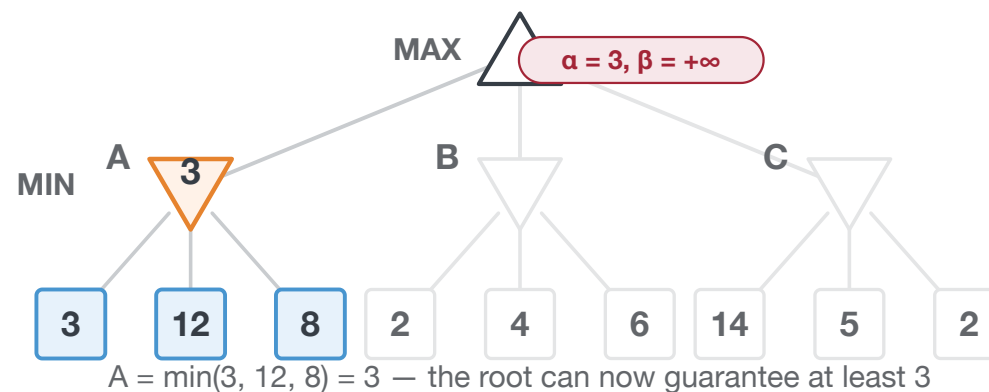
- At a MIN node: if the running minimum drops to or below the inherited α , stop: MAX will never let the game reach here, since MAX already has a branch worth at least α
- At a MAX node: if the running maximum rises to or above the inherited β , stop: MIN will never let the game reach here
- This is exactly “the parent already has a better alternative: don’t bother finishing this branch”



Worked alpha-beta trace: branch A

Starting values: $\alpha = -\infty$, $\beta = +\infty$ (nothing known yet).

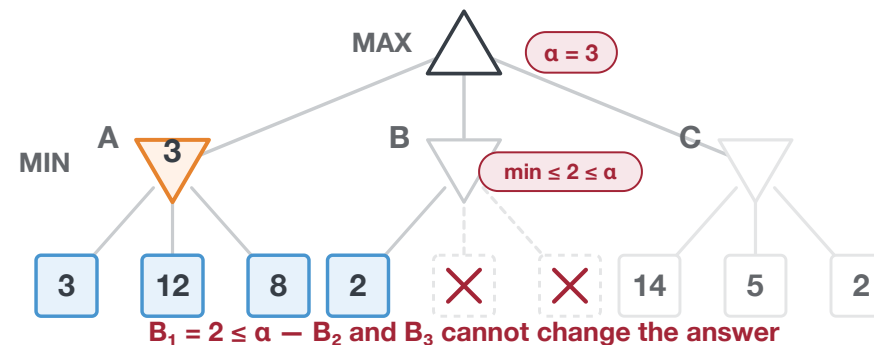
Step	Node visited	Running min at A	Action
1	A1 = 3	3	β at A $\leftarrow 3$ (continue: $3 > \alpha = -\infty$)
2	A2 = 12	3	no change (12 doesn't lower the min)
3	A3 = 8	3	no change; A fully explored, A = 3
-	back at root	$\alpha \leftarrow 3$	root now guarantees at least 3



Worked alpha-beta trace: branch B (pruned)

Root passes $\alpha = 3, \beta = +\infty$ into B.

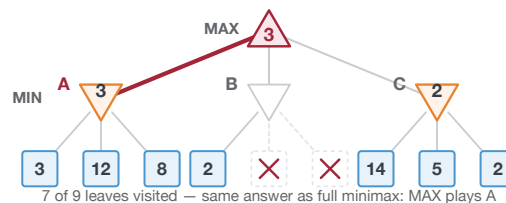
Step	Node visited	Running min at B	Action
4	B1 = 2	2	running min $2 \leq \alpha = 3$: PRUNE
-	B2, B3	never visited	pruned: cannot change the outcome
■	MIN can already guarantee ≤ 2 in branch B, which is worse for MAX than the 3 already banked from A		
■	MAX will never choose B, so its remaining children are provably irrelevant: skip them		



Worked alpha-beta trace: branch C and the final decision

Root still passes $\alpha = 3$, $\beta = +\infty$ into C (B did not raise α , since it never returned an exact value).

Step	Node visited	Running min at C	Action
5	C1 = 14	14	continue
6	C2 = 5	5	continue (still above $\alpha = 3$)
7	C3 = 2	2	C = 2 ; last child anyway
■ Root: $\max(A = 3, B \leq 2, C = 2) = \mathbf{3}$: choose A, exactly matching the full minimax result ■ Total leaves visited: 7 of 9: B2 and B3 were never needed			



Alpha-beta pruning: how much does it help?

- Worst case: no savings, still $O(b^d)$
- With a **good move ordering**: effective branching factor shrinks to about $\sqrt{b}$
 - i.e., search twice as deep in the same time
- Move ordering heuristics (try likely-best moves first) matter as much as the pruning rule itself



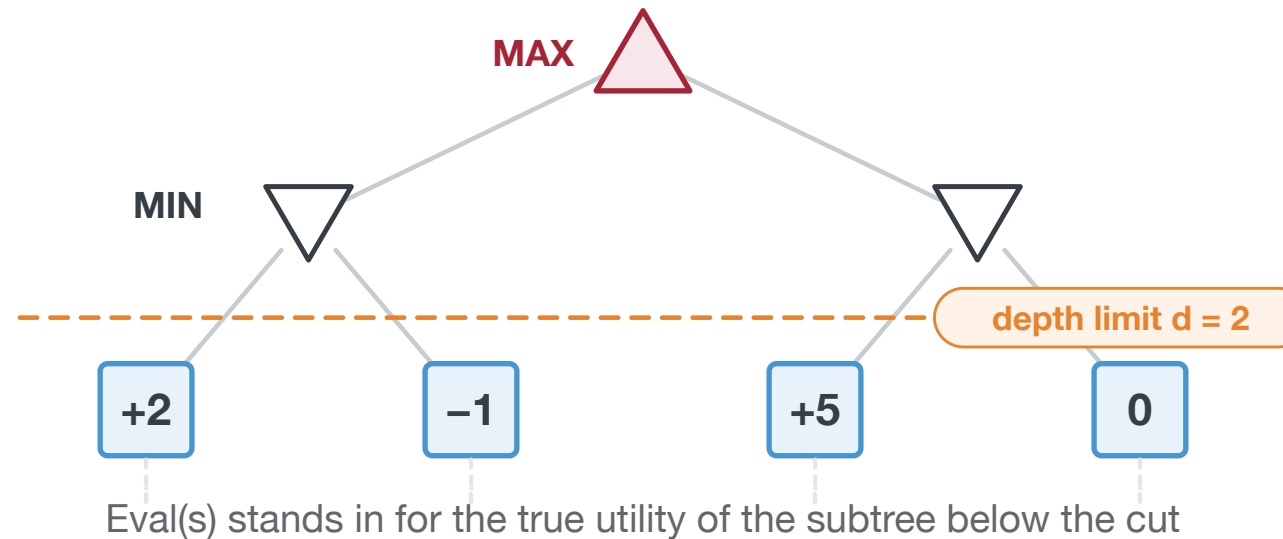
Move ordering heuristics

- Explore the **most promising** move first at each node: the earlier a strong bound is set, the more later siblings get pruned
- Common tricks: previously-found best move at this position (from iterative deepening), captures before quiet moves in chess, killer-move tables, history heuristics
- In our worked example, if C1=14 had been tried before A1=3 and A2=12, more of A and B could have been pruned: ordering is not just cosmetic



Depth-limited search and evaluation functions

- Depth-limited search + an **evaluation function** when we can't reach terminal states
 - e.g., material count in chess as a cheap value estimate
- Still assumes we can **enumerate** moves at each node
- Breaks down when b is enormous (Go: ~ 250) or the game has continuous/huge action spaces



Evaluation functions in practice

- A good $\text{Eval}(s)$ estimates who's ahead **without** searching to the end
- Chess example: material (pawn=1, knight/bishop=3, rook=5, queen=9) plus positional terms (king safety, center control, pawn structure)
- The better Eval is, the shallower you can search and still play well: this is exactly why classical chess engines invested heavily in hand-tuned evaluation



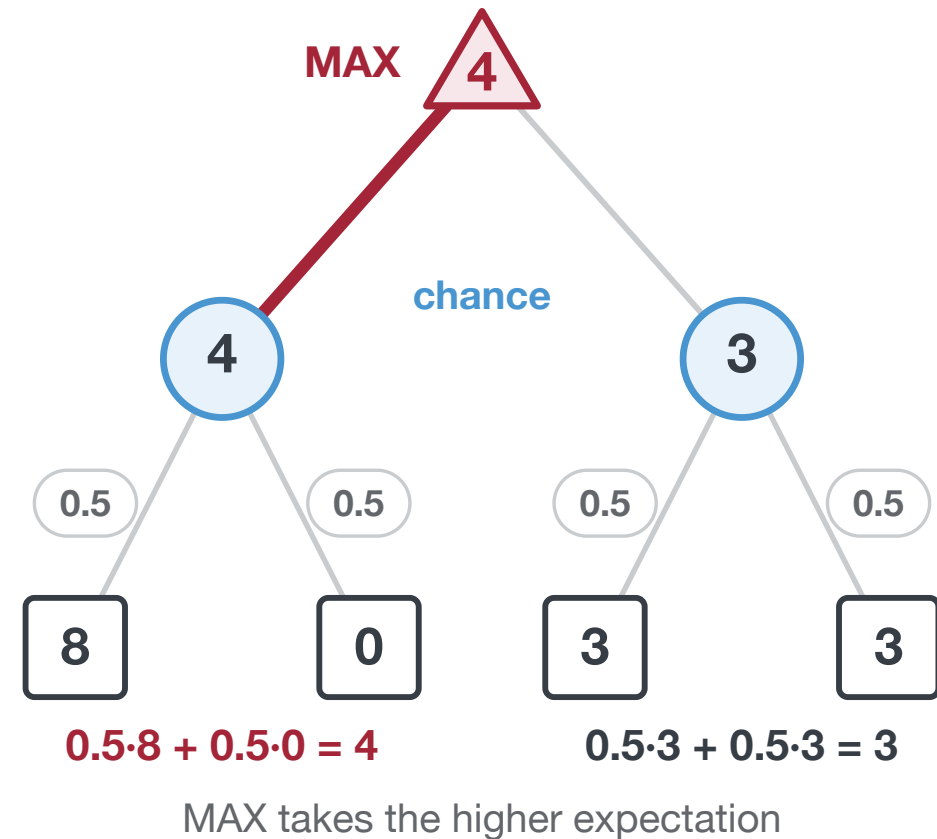
Quiescence and the horizon effect

- Cutting search off at a fixed depth can stop mid-exchange (e.g. right after your queen is captured, one ply before you'd recapture)
- This is the **horizon effect**: a real threat or gain is just past where the search stopped, so Eval misjudges the position
- **Quiescence search**: extend the search past the depth limit along “noisy” lines (captures, checks) until the position settles down



Beyond two-player zero-sum: expectimax

- Some games have **chance** nodes: dice rolls, card draws, shuffled tiles
- Add a third node type alongside MAX and MIN: a **chance node**, whose value is the **expectation** over outcomes, weighted by their probabilities
- $\text{expectimax}(\text{chance node}) = \sum_o P(o) \cdot \text{value}(o)$
- Backgammon is the classic example: this is why early strong backgammon programs looked different from chess engines



Beyond two-player zero-sum: more than two players

- Generalize each ply to be that player's own maximization over **their own** utility
- Players may act independently, or two can gang up against a third: coalitions complicate the picture
- No single “MIN” viewpoint exists once there are 3+ self-interested players



Imperfect information games

- Poker, Bridge, and Diplomacy hide information: opponents' cards, others' plans
- A game tree over **belief states** (probability distributions over what might be true) replaces a tree over known board positions
- Bluffing becomes rational: revealing information through your actions has its own cost, separate from the immediate payoff
- Modern poker AIs (e.g., Pluribus) combine game-theoretic reasoning with search under this uncertainty: a topic beyond today's scope, but built on the same search foundations



Limits of exhaustive game-tree search

- Depth-limited search + evaluation functions get chess-sized games far, but not Go
- Still assumes we can **enumerate** moves at each node and score positions reasonably well
- Breaks down when b is enormous (Go: ~ 250) or a good Eval doesn't exist

This is the gap Monte Carlo Tree Search is built to close.



Adversarial search and game theory

- Minimax is a special case of a much older idea: the **minimax theorem** from game theory (von Neumann, 1928), which guarantees an optimal randomized strategy exists in any finite zero-sum game
- In deterministic, perfect-information games like chess, the optimal strategy against a rational opponent need not be randomized: the classic minimax algorithm suffices
- **Nash equilibrium** generalizes this idea to non-zero-sum and multiplayer settings, relevant once you leave the MAX/MIN template



Symmetry and transposition tables

- The same position can be reached by different move orders: a **transposition**
- A **transposition table** (hash map from position → best known value) avoids re-searching positions already solved elsewhere in the tree
- Board symmetries (rotations, reflections) can similarly collapse equivalent positions into one search node
- Practical engines lean heavily on both to make real move ordering and pruning effective



Check your understanding: minimax & alpha-beta

- Why does alpha-beta return the **same** value as plain minimax, never an approximation?
- In the worked trace, why was branch B pruned but not branch C?
- If C1=14 were explored **before** A1, A2, A3 at the root, would more pruning happen? Why?
- What breaks if a game has chance nodes, and how does expectimax fix it?



Monte Carlo Tree Search

The four MCTS steps

Why not minimax for Go?

- Branching factor ~250, depth of a full game ~150+ moves
- No easy evaluation function for a mid-game Go position (unlike material count in chess)
- Need a way to search **asymmetrically**: spend more effort exploring promising lines, not every line equally



Monte Carlo Tree Search: the idea

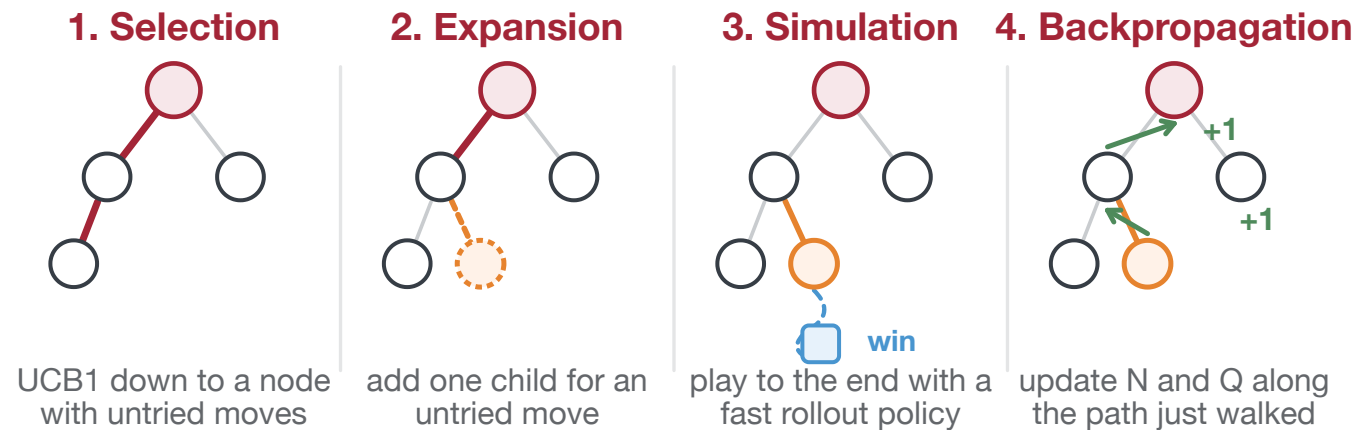
Build the game tree incrementally, guided by random **simulations** (rollouts) rather than exhaustive expansion.

- Grow the tree in the directions that look most promising
- Estimate a node's value by playing out random (or learned) games from it
- Four repeated steps: **selection**, **expansion**, **simulation**, **backpropagation**



The four steps of one MCTS iteration

- **Selection:** walk down from the root by a selection policy, until a node with untried moves
- **Expansion:** add one new child node for an untried move
- **Simulation:** play the game out to the end using a fast policy
- **Backpropagation:** carry the outcome back up the path just walked



Each iteration touches one new leaf and updates statistics along one root-to-leaf path: nothing like minimax's full-tree recursion.

MCTS step 1: Selection

- Starting at the root, repeatedly pick a child according to a **selection policy**
- Balances two goals:
 - **Exploitation**: go toward children with high estimated value
 - **Exploration**: also visit children we haven't tried much
- Continue until reaching a node that is not fully expanded, or a leaf

How do we pick the child? A few slides on, we make this precise with the **UCB1** formula.



MCTS step 2: Expansion

- Add one (or more) new child node(s) to the tree at the selected node
- Represents a move not yet tried from that position
- The tree grows by exactly one node per iteration (in the simplest version)



MCTS step 3: Simulation (rollout)

- From the new node, play out the rest of the game using a fast, simple policy
 - Classically: uniformly random moves to the end
 - Modern variants: use a learned policy network instead
- Record the outcome (win/loss, or a score)



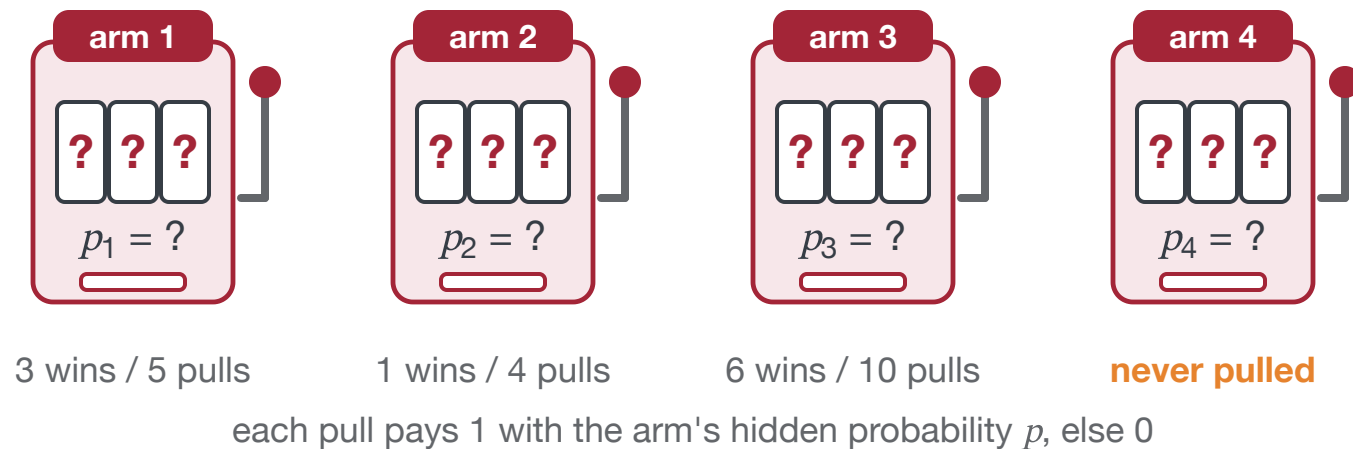
MCTS step 4: Backpropagation

- Propagate the simulation's result back up every node visited in this iteration
- Update each node's:
 - visit count N
 - average value Q
- Repeat steps 1-4 many times (thousands to millions of iterations); more iterations = a better estimate



Multi-armed bandits and the exploration-exploitation tradeoff

- Imagine k slot machines (“bandit arms”) with unknown payout rates. Which do you pull to maximize total winnings over many pulls?
- Pulling only the arm that looks best so far risks missing a better arm you haven’t tried enough
- Pulling arms uniformly at random wastes pulls on arms you already know are bad
- Selecting a child in MCTS is exactly this problem, once per node, at every level of the tree



UCB1: the selection formula

To pick the child during selection, treat it like a **multi-armed bandit** problem.

- **UCB1** score for a child:

$$Q + c\sqrt{\frac{\ln N_{\text{parent}}}{N_{\text{child}}}}$$

- Q : average value so far (exploitation term)
 - second term grows for **under-visited** children (exploration term)
 - c : constant controlling exploration strength
- A child never visited ($N_{\text{child}} = 0$) has an infinite UCB1 score: untried children are always tried first

Worked MCTS example: the setup

We'll trace 4 iterations by hand from a root with two first moves, **L** and **R**, using UCB1 with $c = 1$. Win = 1, loss = 0.

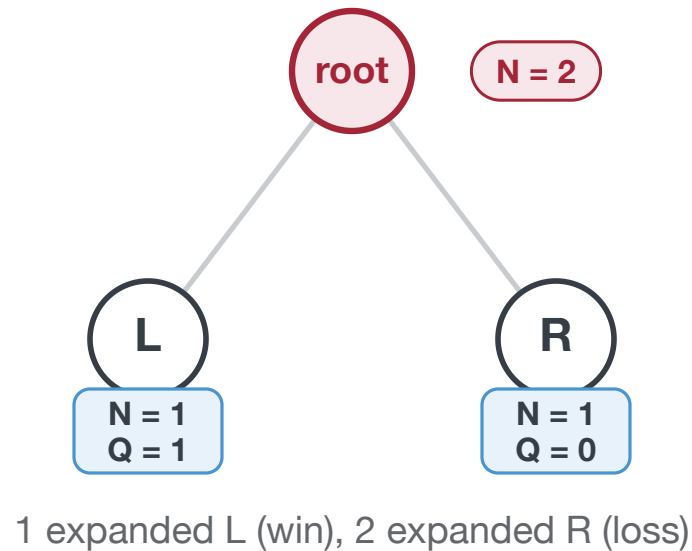
- Root: $N = 0$ initially, children L and R both untried
- Convention: expand every untried child of a node once before using UCB1 to choose among fully-expanded children



Worked MCTS trace: iterations 1-2

Iteration	Selection	Expand	Rollout result	Update
1	root has untried child L	expand L	win (1)	$N(L)=1$, $Q(L)=1$; $N(\text{root})=1$
2	root has untried child R	expand R	loss (0)	$N(R)=1$, $Q(R)=0$; $N(\text{root})=2$

■ Both children now expanded once; from here on, selection at the root uses UCB1

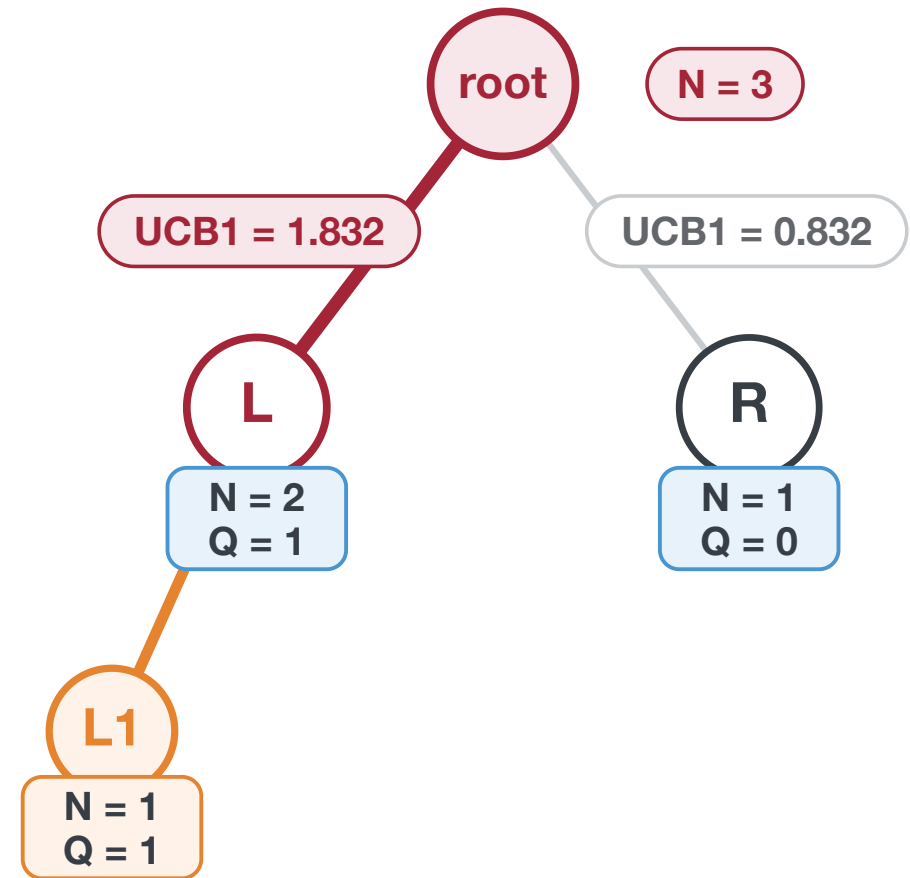


Worked MCTS trace: iteration 3

Root is fully expanded ($N_{\text{root}} = 2$).

Compute UCB1 for each child, $c = 1$:

- $\text{UCB1}(L) = 1 + 1 \cdot \sqrt{\ln 2 / 1} = 1 + 0.832 = \mathbf{1.832}$
- $\text{UCB1}(R) = 0 + 1 \cdot \sqrt{\ln 2 / 1} = \mathbf{0.832}$
- Select **L** (higher score). L has untried children L1, L2: expand **L1**
- Rollout from L1: win (1). Update: $N(L1)=1$, $Q(L1)=1$; $N(L)=2$, $Q(L)=(1+1)/2=1$; $N(\text{root})=3$



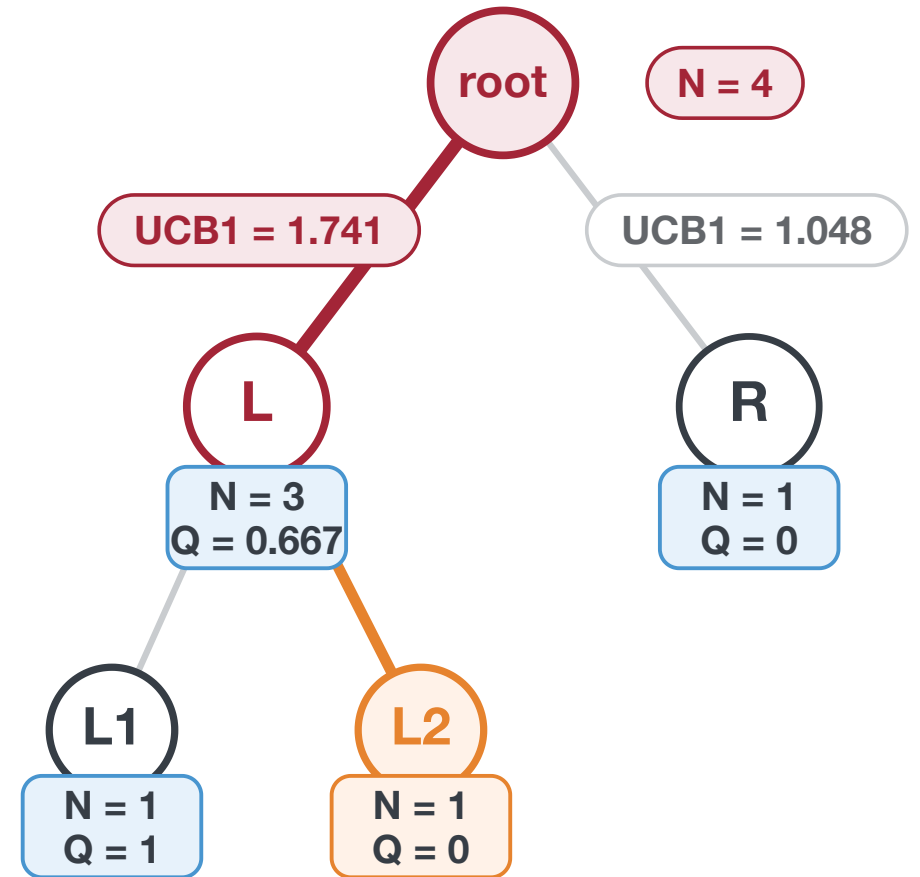
L wins on UCB1; L1 expanded, rollout wins

Worked MCTS trace: iteration 4 and the final choice

Root: $N_{\text{root}} = 3$. Compute UCB1 again:

- $\text{UCB1}(L) = 1 + 1 \cdot \sqrt{\ln 3/2} = 1 + \sqrt{0.549} = 1 + 0.741 = \mathbf{1.741}$
- $\text{UCB1}(R) = 0 + 1 \cdot \sqrt{\ln 3/1} = \sqrt{1.099} = \mathbf{1.048}$
- Select **L** again. L's untried child L2 gets expanded; rollout from L2: loss (0)
- Update: $N(\text{L2})=1$, $Q(\text{L2})=0$; $N(\text{L})=3$, $Q(\text{L})=(1+1+0)/3=0.667$; $N(\text{root})=4$

Child	N	Q
L	3	0.667
R	1	0



final choice: L — highest N and highest Q

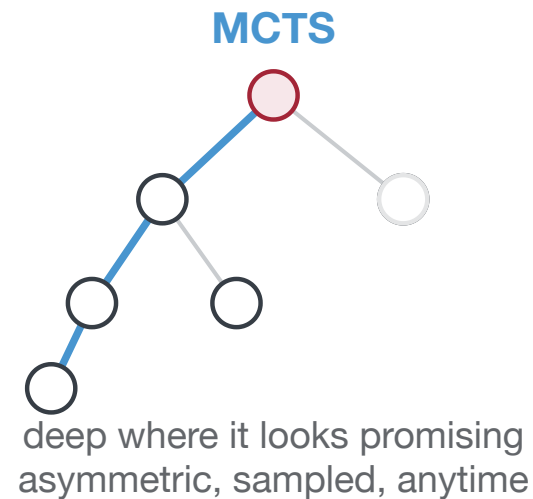
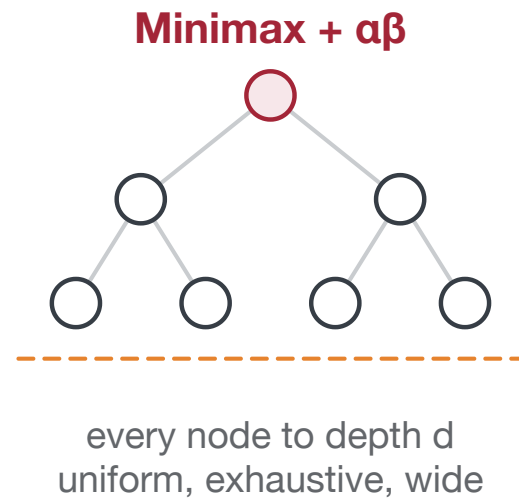
Choosing the final move: max child vs. most-visited child

- After the iteration budget is spent, MCTS must commit to one move at the root
- **Max-child:** pick the highest Q : risky, since a rarely-visited child can have a lucky, noisy Q
- **Robust child (most-visited):** pick the highest N : the standard choice, since UCB1 naturally visits genuinely strong moves more often
- In our worked example, both criteria happen to agree: **L**, with $N=3$ and the higher Q



MCTS vs. minimax

Aspect	Minimax (+ $\alpha\beta$)	MCTS
Coverage	Exhaustive to a depth	Asymmetric, guided by simulation
Needs eval function?	Yes (if depth-limited)	Not strictly: rollouts substitute
Scales to Go-sized b ?	Poorly	Yes
Anytime?	Only with iterative deepening	Yes: more iterations, better answer



From random rollouts to learned rollout policies

- Purely random rollouts are unbiased but noisy and slow to converge: most random Go games are nonsensical
- Replacing the random rollout policy with a **fast learned policy** (even a small one) sharply improves the quality of each simulation's signal
- This is the seed of the idea AlphaGo takes to its logical extreme later in this lecture



Practical considerations: rollout policy quality

- A better rollout/simulation policy gives lower-variance value estimates per iteration: fewer iterations needed for the same confidence
- But a better policy usually costs more compute per rollout: a budget tradeoff, not a free win
- Domain knowledge (e.g., “don’t play obviously illegal-looking Go moves” heuristics) can cheaply improve rollout quality



Practical considerations: iteration budget and anytime behavior

- MCTS can be stopped after **any** number of iterations and still return a legal, reasonable move: this **anytime** property fits real time controls (chess clocks, move-time budgets)
- More iterations monotonically tend to improve move quality, but with diminishing returns
- Contrast with minimax: cutting it off mid-search (before finishing a ply) doesn't give a well-defined answer without extra bookkeeping



Practical considerations: parallelizing MCTS

- Multiple rollouts can run concurrently from the same tree, if care is taken about shared statistics (locking, or approximate updates)
- **Leaf parallelization:** many rollouts from the same leaf in parallel
- **Root parallelization:** run several independent trees, then combine their statistics at the end: simple and used in some production systems
- Both trade some search efficiency for wall-clock speed on multi-core or multi-machine hardware



MCTS in non-game domains

- The four-step template (select/expand/simulate/backpropagate) generalizes past games: automated planning, scheduling, and combinatorial optimization all use MCTS variants
- Anywhere you have a large branching decision tree and a way to “simulate to an outcome,” MCTS is a candidate
- This generality is part of why MCTS became central well beyond Go



Check your understanding: MCTS

- Why does an unvisited child always get selected first under UCB1?
- In our worked trace, why did the algorithm keep choosing to expand under L instead of trying R again?
- What's the difference between choosing the final move by highest Q vs. highest N, and why does the field prefer N?
- Why is MCTS described as “anytime” while plain minimax generally isn't?





Case study: AlphaGo and AlphaZero

Search meets learning

The problem AlphaGo solved

- Go: branching factor ~ 250 , no good handcrafted evaluation function
- Pure MCTS with random rollouts was too weak to reach human-expert strength
- AlphaGo's fix: replace **parts** of MCTS with learned neural networks



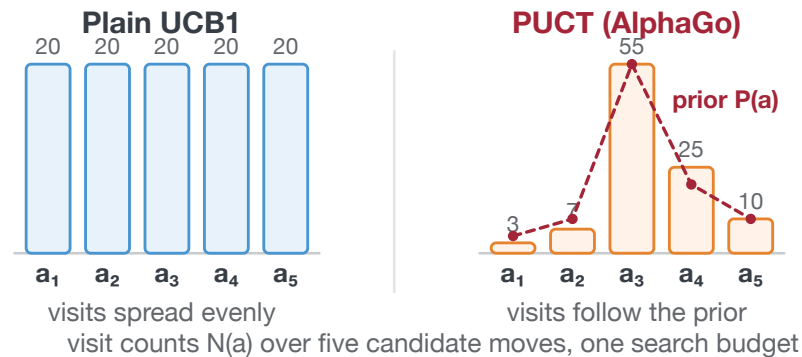
AlphaGo's architecture: policy and value networks

- **Policy network:** predicts a probability distribution over promising moves: narrows the **selection/expansion** step
- **Value network:** predicts the probability of winning from a position: replaces/supplements random **rollouts**
- MCTS still runs the tree search; the networks make it look in the right places instead of everywhere equally



How the networks change tree search: PUCT

- Plain UCB1's exploration term treats every child as equally worth trying until visited
- AlphaGo instead weights exploration by the policy network's prior probability $P(a)$ for each move:
$$\text{PUCT} = Q + c P(a) \frac{\sqrt{N_{\text{parent}}}}{1 + N_{\text{child}}}$$
- Moves the policy network considers implausible are explored far less, freeing search budget for moves it considers promising
- The value network's estimate can substitute for (or blend with) a random rollout at each new leaf



Training AlphaGo: supervised phase

- Policy network initially trained via supervised learning on a large dataset of human expert games
- Objective: predict the move a strong human player would make from a given board position
- Gives the network a reasonable prior before any self-play begins: bootstraps the search instead of starting from nothing



Training AlphaGo: reinforcement learning phase

- Policy network then refined with self-play reinforcement learning: play against past versions of itself, update toward moves that led to wins
- Value network trained on outcomes from these self-play games: regress the eventual winner from the board position
- This RL fine-tuning phase is what pushed the policy past merely imitating human play



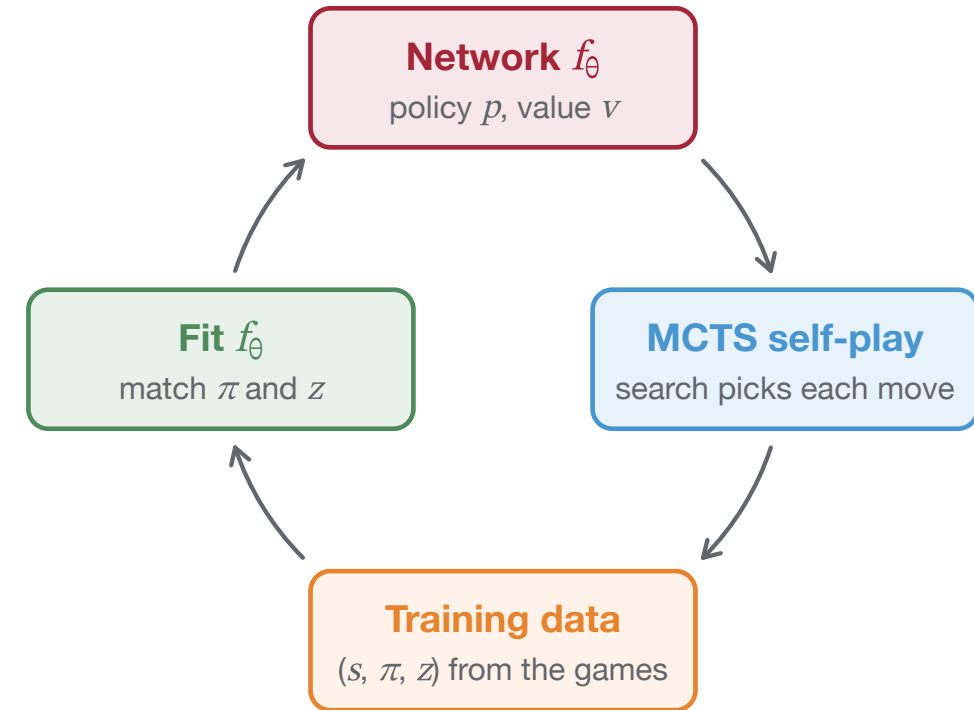
AlphaZero: dropping the training wheels

- No human game data at all: learns purely from self-play, starting from random play
- No handcrafted features: just the board state as input
- One architecture, one training loop, applied to Go, chess, and shogi with superhuman results in all three



The self-play training loop

- Current network plays games against itself, guided by MCTS
- Game outcomes + MCTS's visit-count distributions become training targets
- Network is retrained to better predict both the **move** (policy) and the **outcome** (value)
- Repeat: an improving loop with no external data needed



nothing enters from outside: no human game data

AlphaGo vs. AlphaGo Zero vs. AlphaZero: what changed

Version	Training data	Handcrafted features?	Games
AlphaGo	Human games + self-play	Some	Go only
AlphaGo Zero	Self-play only	None	Go only
AlphaZero	Self-play only	None	Go, chess, shogi

- Each step removed one more piece of human-supplied scaffolding, while performance kept improving



AlphaZero's generality

- The same architecture and training loop (no game-specific tuning) reached superhuman strength in three structurally different games
- Chess (no ko-like repetition rules, check/checkmate) and shogi (piece drops after capture) differ substantially from Go, yet none needed bespoke features
- This generality is itself evidence that “search scaffold + learned priors” is a broadly reusable recipe, not a Go-specific trick



Why neural-guided search beats either half alone

- Neural networks alone (no search): fast, pattern-matching intuition, but no lookahead: can miss forced tactical sequences
- Search alone (no networks): correct lookahead, but needs either an impractically large budget or a weak evaluation, as plain MCTS showed for Go
- Together: the network proposes **where** to look and **how good it looks**; search verifies and refines that intuition through lookahead

This is the course’s “search meets learning” thesis in its purest form: MCTS provides the search scaffold, neural networks provide the pattern recognition that makes the search tractable.



Check your understanding: AlphaGo & AlphaZero

- What two roles do the policy and value networks play inside MCTS?
- Why did dropping human game data (AlphaGo → AlphaGo Zero) not hurt performance?
- Concretely, what does the PUCT formula do differently from plain UCB1?
- Why is a policy network alone (with no search) not enough to play at superhuman strength?



SAT solvers and constraint satisfaction

Boolean satisfiability · Constraint satisfaction

Boolean satisfiability (SAT)

Given a Boolean formula, is there **some** True/False assignment of its variables that makes it true?

- The standard form is an AND of **clauses**, where each clause is an OR of variables or their negations
- Example: $(x_1 \vee \neg x_2) \wedge (x_2 \vee x_3) \wedge \neg x_1$
 - Satisfied by $x_1 = \text{False}$, $x_2 = \text{False}$, $x_3 = \text{True}$
- With n variables there are 2^n assignments, so trying them all fails fast



Why SAT matters

- SAT is **NP-complete**: the canonical hard search problem
- Many real problems reduce to SAT: circuit verification, scheduling, planning, puzzle solving
- A fast SAT solver gives you a fast solver for anything you can encode as clauses
- Modern solvers routinely handle real-world formulas with millions of variables, because real instances have structure that worst cases lack



Constraint satisfaction problems (CSPs): the general view

SAT is a special case of a more general framework.

- A CSP has **variables**, each with a **domain** of possible values, and **constraints** relating them
- SAT: domains are `{True, False}`, constraints are clauses
- Goal: find an assignment to every variable that satisfies every constraint



CSPs: examples beyond SAT

- **Map coloring:** variables = regions, domain = {colors}, constraint = adjacent regions differ
- **N-queens:** variables = queen columns, domain = {rows}, constraint = no two share a row/diagonal
- **Scheduling:** variables = tasks, domain = {time slots}, constraints = no resource conflicts, deadlines respected
- **Sudoku:** variables = cells, domain = {1, ..., 9}, constraint = no repeats in a row, column or box



Solving SAT and CSPs: the big idea

- **Backtracking search:** assign one variable at a time, and back up as soon as a constraint is violated
- **Propagation:** after each assignment, cross off values that can no longer work, so dead ends show up early
- **Ordering:** branch first on the variable with the fewest legal values left: if it is going to fail, fail fast

The pattern repeats across this course: search + smart pruning, whether the domain is Boolean, board positions, or CSP variables.



Check your understanding: SAT & CSPs

- Why does brute-force SAT stop being practical, and what does a solver do instead?
- How would you phrase Sudoku as a CSP: what are the variables, domains and constraints?
- How does branching on the most-constrained variable relate to alpha-beta's move-ordering idea?



Wrapping up: search meets learning

Recap · Looking ahead

A historical timeline of game-playing AI milestones

Year	Milestone	Core technique
1997	Deep Blue beats Kasparov (chess)	Minimax + alpha-beta + handcrafted eval
2011	Watson wins Jeopardy!	Search + learned language understanding
2016	AlphaGo beats Lee Sedol (Go)	MCTS + learned policy/value networks
2017	AlphaZero masters chess/shogi/Go	Pure self-play, one architecture



Comparing today's four search techniques

Technique	What it prunes/guides with	Where it shines
Minimax + alpha-beta	Provable bounds (α, β)	Small-to-medium b , need exact optimal play
MCTS	Sampled simulations + UCB1	Huge b , no good handcrafted eval
Neural-guided search	Learned priors from data	Huge b and no handcrafted eval exists
SAT / CSP solvers	Backtracking + constraint propagation	Boolean/combinatorial satisfaction



Wrapping up: search meets learning

- Minimax/alpha-beta and SAT/CSP solvers: search techniques that prune based on **guarantees**
- MCTS: search guided by **sampling**, generalizes where exhaustive search can't
- AlphaGo/AlphaZero: learning supplies the **priors** that make search tractable at real-world scale



Where these ideas show up in your course project

- Any project involving a large discrete decision space (planning, scheduling, itinerary optimization, puzzle generation) can likely be framed with the search tools from today
- If handcrafted evaluation is hard but you can simulate outcomes, consider an MCTS-style approach before reaching for a full learned agent
- If your problem is really a constraint problem in disguise (assignment, coloring, matching), a CSP/backtracking framing may be simpler than either search extreme



Looking ahead

Assignment 2 (Unsupervised learning) is due today, and **Assignment 3** (Search) is released (due October 15, 3pm). Next week: **Language models and inference**: where the “learning” side of this course takes center stage.

- Search returns explicitly in Lecture 9-10 (reinforcement learning), where the exploration-exploitation ideas from MCTS reappear as the core of the RL problem itself





THANK YOU

Stevens Institute of Technology
1 Castle Point Terrace, Hoboken, NJ 07030